Mini Project - 2

MPI Performance Modeling: SimGrid and LogGOPS Analysis

Objective

Study and analyze the difference between simulated performance and actual execution of MPI applications using performance modeling tools.

Part A: Analysis

- Use the predictions already generated in Mini Project-1
- Compare:
  - Actual execution time
  - Simulated execution time (from Mini Project-1 using):
    - SimGrid
    - LogGOPSim
- Both tools are open-source (links in slides), so you can:
  - Explore their source code
  - Understand how they model communication and computation
  - Investigate why differences arise in both tools

Part B: Improvement Strategies

Propose and validate strategies to improve prediction accuracy such that the predicted results are as close as possible to the actual execution results.

Output

- Proposed strategies
- Validation showing improvement (before vs after comparison)

1 SimGrid Analysis

1.1 How SimGrid Works: Emulated Computation + Simulated Communication

1.1.1 Computation is Emulated (Executed)

The mathematical operations, loops, and memory accesses in your code are executed *for real* on our physical CPU. SimGrid intercepts the execution time (benchmarks the CPU cycles) and injects this time into the simulation's virtual clock.

1.1.2 Communication is Simulated

When our code hits an MPI call (e.g., `MPI_Bcast`, `MPI_Send`), SimGrid intercepts it. No actual data is sent over a real network card. Instead, SimGrid copies the data within the system's memory and uses mathematical models to calculate exactly how long that transfer *would* take on the virtual network we defined, advancing the virtual clock accordingly.

1.2 Network Topology

The network topology is defined in the XML configuration file:

```
<?xml version='1.0'?>
<!DOCTYPE platform SYSTEM "https://simgrid.org/simgrid.dtd">
<platform version="4.1">
  <zone id="world" routing="Full">
    <cluster id="litepi-crossbar"
             prefix="node"
             radical="1-10"
             suffix=""
             speed="25.73Gf"
             bw="117.04MBps"
             lat="77.18us"/>
  </zone>
</platform>
```

With this `<cluster>` tag, SimGrid automatically generates a dedicated private link for each of the 10 nodes (node1 through node10). Every single node has its own personal connection to the central switch with a full **117.04 MBps** capacity and **77.18 µs** latency.

1.3 Observation 1: Computation Time Impact

Taking computation time between MPI Calls = 0 does not affect the predictions. This is because in a pure Two Tree broadcast code, the only computations are simple pointer arithmetic and offset calculations, which only take nanoseconds. Using

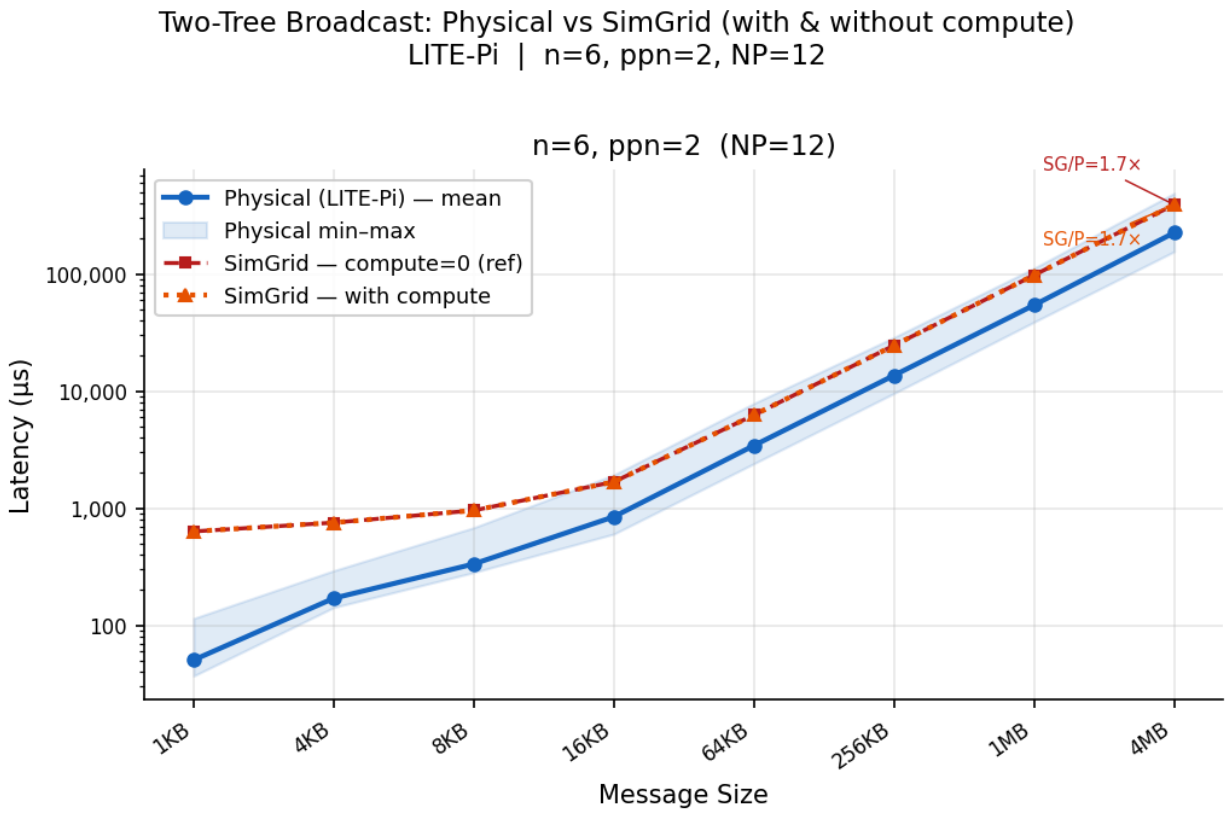


Figure 1: Impact of computation time on predictions

`-cfg=smpi/simulate-computation:0` flag gives a very small improvement (0.5%) in timings.

1.4 Observation 2: The Two-Hop Latency Trap

1.4.1 End-to-End vs. Link-Level Latency

There is a critical mapping issue between empirical data and SimGrid's `<cluster>` tag topology.

When we run an `osu_latency` test on physical hardware, the result (77.18 µs) is the **end-to-end** delay between 2 nodes. However, the `lat` attribute in SimGrid's `<cluster>` tag does not define end-to-end latency; it defines the latency of a **single link** from a node to the switch.

The Double-Count: Because the crossbar cluster creates a star topology, a message from Node 0 to Node 1 must travel *up* the link to the switch, and *down* the link to the receiver.

The Error: If you set the link latency to 77.18 µs, SimGrid calculates: 77.18+77.18 = 154.36 µs total delay per message.

The Fix: By dividing the empirical end-to-end measurement by two (38.59 µs), you ensure that the uplink + downlink traversal in the simulation perfectly equals your real-world 77.18 µs benchmark.

1.5 Performance Comparison

1.5.1 Results on Our Cluster

Table 1: Comparison of Physical vs. Simulated Execution Times

Config	Phys (µs)	lat=38.59µs	lat=77.18µs	Ratio (38µs)	Ratio (77µs)
n=6,p=1	102,340	288,557	461,476	**2.82**×	4.51×
n=6,p=2	224,797	390,064	606,383	**1.74**×	2.70×
n=6,p=4	317,558	677,449	936,997	**2.13**×	2.95×
n=8,p=1	108,556	305,236	478,325	**2.81**×	4.41×
n=10,p=4	383,860	940,625	1,286,632	**2.45**×	3.35×

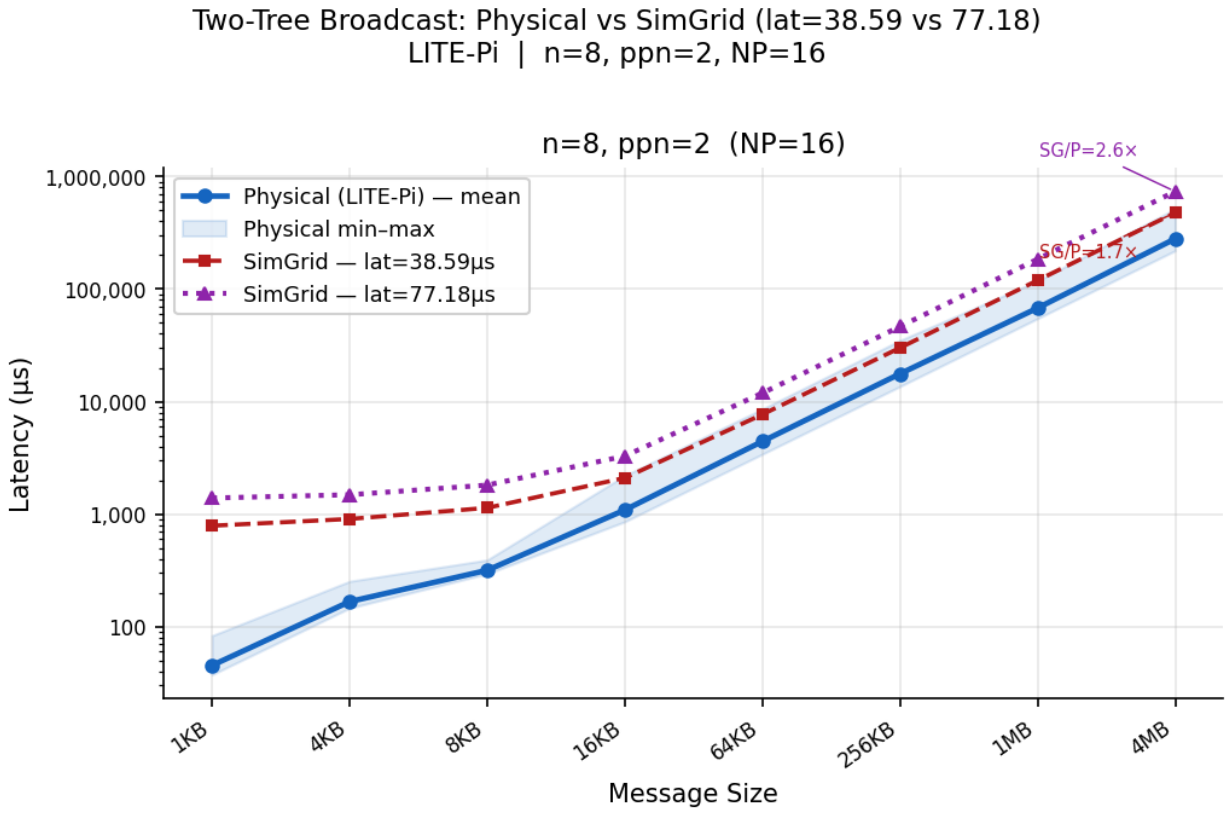


Figure 2: Graphical comparison of execution times on our cluster

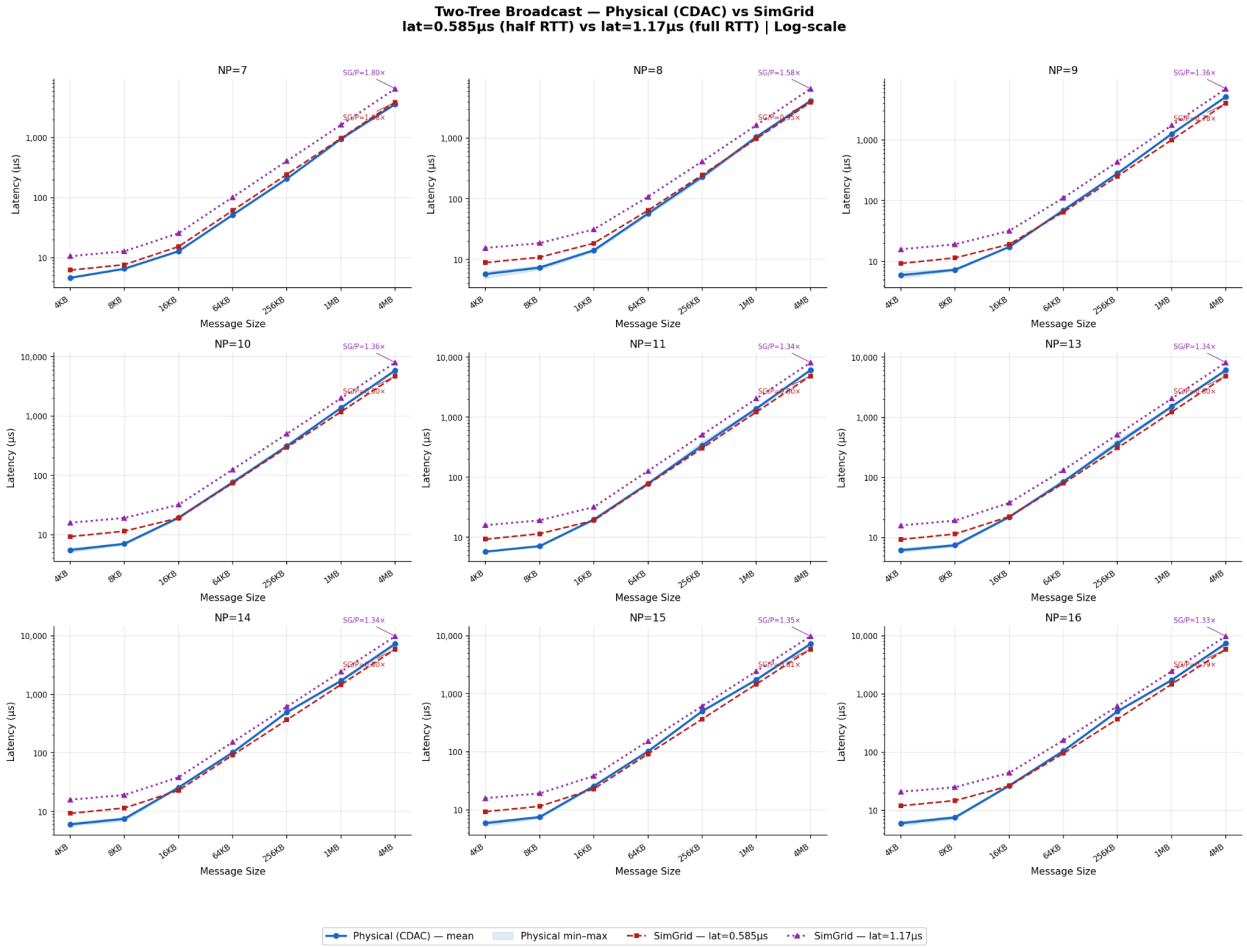


Figure 3: Detailed performance analysis on our cluster

1.5.2 Results on CDAC Cluster

Table 2: CDAC Cluster Performance Comparison

NP	Phys (µs)	lat=0.585µs	lat=1.17µs	Ratio (0.58µs)	Ratio (1.17µs)
7	206.2	244.3	408.1	**1.18**×	1.98×
8	230.6	247.5	413.9	**1.07**×	1.79×
9	282.0	252.2	436.5	**0.89**×	1.55×
10	312.5	295.9	500.7	**0.95**×	1.60×
11	339.0	305.3	510.1	**0.90**×	1.50×
14	493.4	366.4	612.1	**0.74**×	1.24×
16	500.9	369.6	617.9	**0.74**×	1.23×

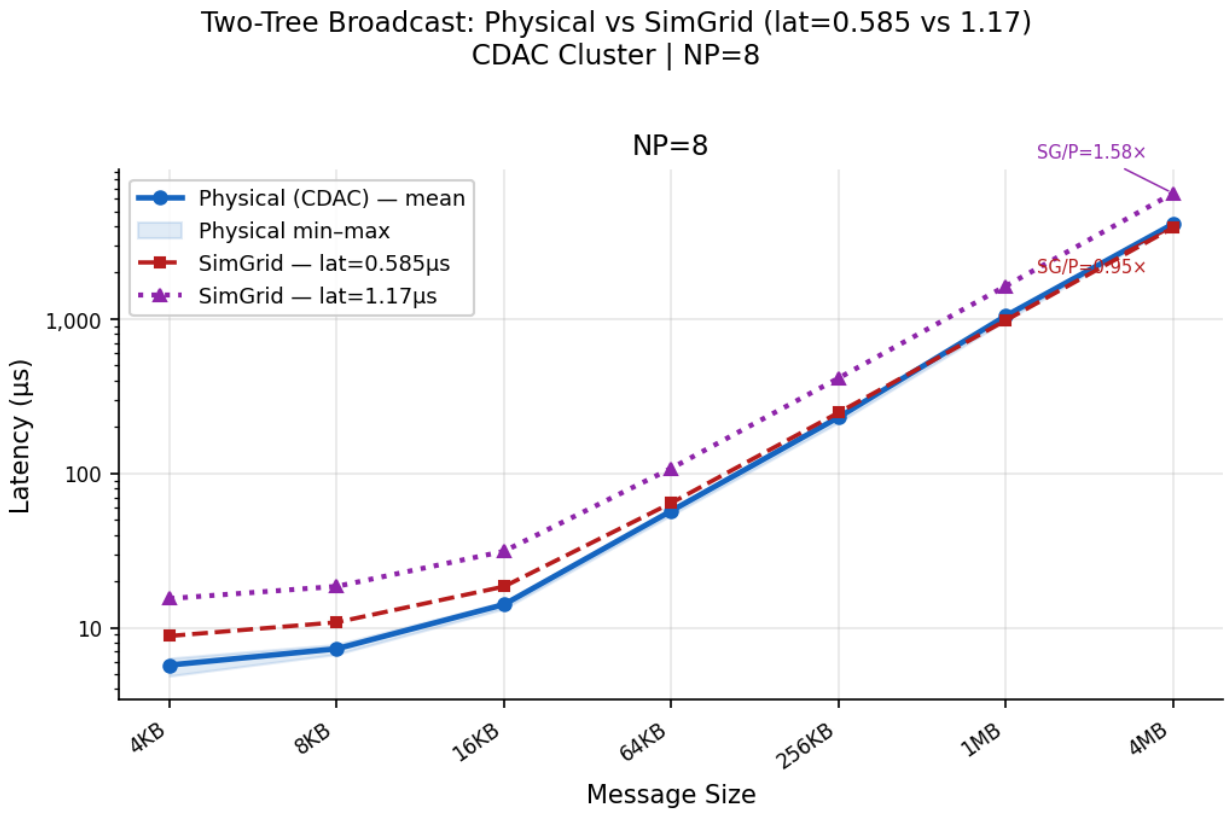


Figure 4: CDAC cluster performance visualization

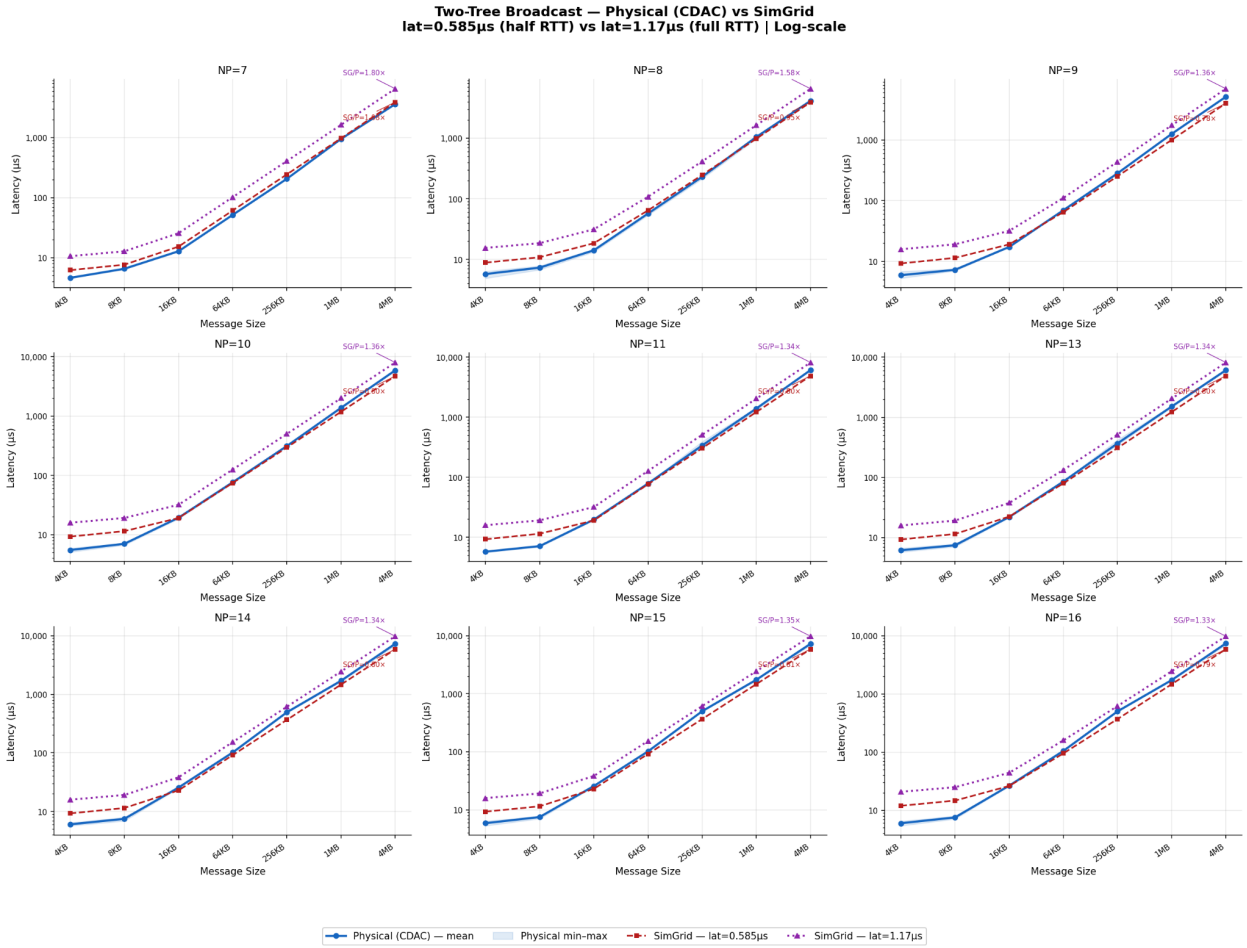


Figure 5: Detailed performance analysis on CDAC cluster

1.6 Why Different Results for Different Clusters?

1.6.1 The Network Flow Model Effect

LITE-Pi (Ethernet): Has very low bandwidth (117 MBps). For a 4MB message, the time taken is almost entirely dictated by bandwidth (transfer time). When multiple processes try to broadcast 4MB simultaneously, they severely bottleneck at the switch. SimGrid uses a "Max-Min Fairness" mathematical model to divide bandwidth among competing flows. When 10 Raspberry Pis blast the network simultaneously, SimGrid aggressively throttles all the flows, causing a massive simulated slowdown (the $\sim$2.8$\times$ overestimate).

CDAC (InfiniBand): Has astronomically high bandwidth (12,013 MBps). The time taken to transfer a message is much shorter and largely dictated by the base network latency (1.17µs), not the bandwidth. Because the bandwidth is so massive, the InfiniBand links almost never become saturated by the Two-Tree broadcast. Since there is no bandwidth bottleneck, SimGrid doesn't throttle the flows, and its predictions remain perfectly accurate.

Conclusion: In short, SimGrid's built-in fairness algorithms end up over-correcting for the Ethernet congestion, resulting in simulated times that are much slower than the real physical cluster.

2 LogGOPS Analysis

2.1 Introduction to the LogGOP Model

The LogGOP model is a mathematical framework used to predict the performance of parallel algorithms (like MPI collective operations) across a network. It is an extension of the classic LogP model, explicitly adding support for large messages.

When you use LogGOPSim, the simulator takes your network's parameters and a "schedule" of operations (.goal files) and mathematically plays out the timeline of exactly when every single processor and network card is busy or idle.

2.2 The LogGOP Parameters

The name "LogGOP" comes from the parameters that define the network and hardware characteristics:

L (Latency): The time it takes for a single byte to travel physically through the network switch from the source Network Interface Card (NIC) to the destination NIC.

o (Overhead): The fixed amount of time the CPU spends preparing a message to be sent or processing a message that was just received. During this time, the CPU is completely locked and cannot do any other work.

g (Gap): The minimum time interval the hardware requires between sending two consecutive small messages. This dictates the maximum message injection rate of the NIC.

G (Gap per byte): The time it takes to push one single byte into the network. This represents the reciprocal of the network bandwidth (e.g., higher bandwidth = smaller G).

O (Overhead per byte): The time the CPU spends copying/processing one single byte of a large message (e.g., copying from application memory to network buffers).

P (Processors): The total number of nodes/ranks participating.

2.3 How LogGOPSim Simulates an Operation

LogGOPSim acts as a discrete-event simulator. It reads .goal files, which contain lists of events for each rank (calc, send, recv), and tracks two "clocks" for every single node:

- **CPU Clock:** When is the processor free to do the next task?
- **NIC Clock:** When is the network card free to inject the next message?

2.3.1 Timeline of a Single Send Operation

Let's examine the timeline of a single send of size S from Host A to Host B:

Step 1: Sending (Host A)

- **CPU Phase:** Host A's CPU spends time packaging the message. It is locked for $o + (S \times O)$ time.
- **NIC Phase:** The CPU hands the message to the NIC. The NIC starts pushing bytes over the wire. The NIC is locked for $g + (S \times G)$ time. Host A cannot send its next message until this time passes.

Step 2: In Transit The message travels across the physical network. The final byte arrives at Host B's NIC exactly at time: (Time the NIC finished pushing the message) + L.

Step 3: Receiving (Host B) Host B's CPU wants to read the message. It can only start reading when two conditions are met:

1. The message has fully arrived at its NIC.
2. Host B's CPU is not busy doing something else (like computing a calc node or reading a previous message).

Once both are true, Host B's CPU locks for $o+(S\times O)$ to read the message into its application memory.

2.4 Why the Predictions Are Fast

LogGOPSim steps through every single dependency in the .goal file until the very last recv finishes on the very last node. The time on that final node's clock is outputted as the Predicted Execution Time.

This prediction is essentially a **mathematical lower-bound**. LogGOPSim assumes:

- The network switch has infinite capacity (no congestion or dropped packets)
- The OS never interrupts the MPI process to run background tasks (no OS jitter)
- The memory bus has infinite bandwidth

2.5 NetGauge Parameters

2.6 Performance Analysis and Plots

2.6.1 CDAC Cluster Results

Observation: The predicted execution time is consistently faster (smaller) than the measured execution time.

Example (N=8, M=4MB): Measured = 4.13 ms, Predicted = 2.78 ms.

Parameter	Pi-Cluster Value	CDAC Value
Latency (L)	81.999 µs	1.503 µs
CPU Overhead (o)	6.678 µs	0.203 µs
Per-byte overhead (O)	0.0026 µs/byte	0.000024 µs/byte
Gap (g)	147.22 µs	0.216 µs
Per-byte gap (G)	0.008 µs/byte	0.0001 µs/byte

Figure 6: NetGauge parameter measurements

Why it differs: This represents a perfectly healthy and expected simulation! LogGOPSim calculates a theoretical mathematical lower bound. It assumes a perfect, contention-free network where messages never collide at a switch, and it ignores OS jitter, context switching, and the MPI software stack overhead. Because the CDAC Netgauge parameters were accurate (L=1.5µs), the theoretical model safely serves as an optimal baseline that real hardware will trail by ∼30-50% due to real-world friction.

2.6.2 LITE-Pi Cluster Results

Observation: The predicted execution time is drastically slower (larger) than the measured execution time for almost all sizes.

2.7 Analysis of LITE-Pi Cluster Discrepancies

2.7.1 The "Ping-Pong" Parameter Fallacy

The parameters provided for the Pi Cluster (L=81.99µs, o=6.67µs, g=147.22µs, G=0.008µs/B) were likely derived using a standard Netgauge point-to-point ping-pong test.

The Reality: A ping-pong test measures the absolute latency of waking up the CPU, traversing the OS kernel network stack, hitting the NIC, and waiting for the receiver to do the same. It forces a complete flush of the pipeline on every message.

The Simulation: The traces simulated are for a pipelined collective operation (like a Two-Tree Broadcast). In a pipeline, the system enters a "steady state" where data is streaming. The Linux kernel uses TCP windowing, Jumbo frames, and DMA offloads to stream this data smoothly. The real effective gap (g) and overhead (o) approach zero during a steady stream.

The Result: LogGOPSim rigidly applies the massive 147.22µs gap and 81.99µs latency to every single chunk sent in the pipeline, creating massive artificial delays that don't exist in the real, optimized streaming hardware.

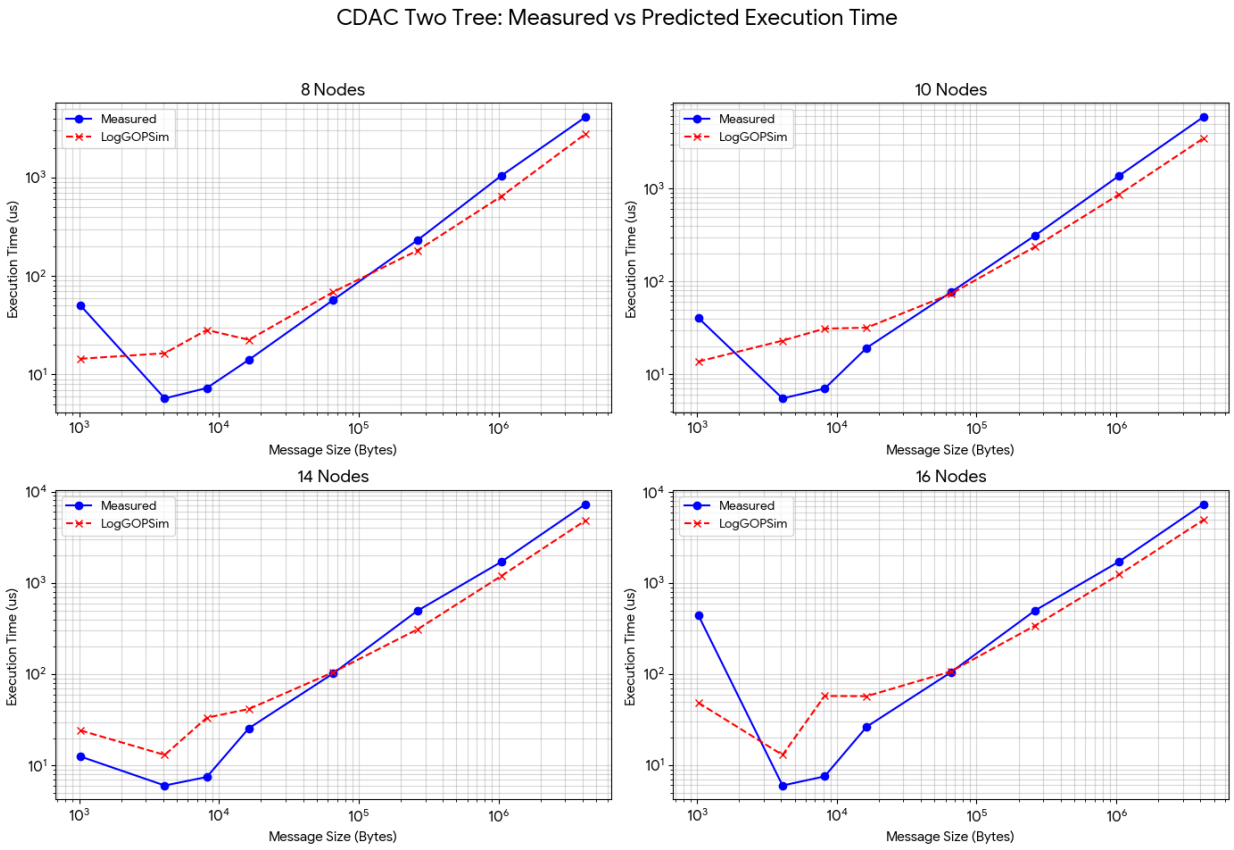


Figure 7: LogGOPS predictions vs. measured times on CDAC cluster

2.7.2 Extreme Overestimation for Small Messages

Look at the small message sizes (e.g., N=6, P=1, M=1024):

- Measured: 28.60 µs
- Predicted: 362.63 µs (∼12× higher)

For small messages, the time is entirely dominated by the static parameters ($L + 2o + g$). Just adding your parameters ($81.99 + 2(6.67) + 147.22$) gives roughly 242 µs for a single theoretical hop. Because the collective algorithm requires multiple hops and Waitall synchronizations, the simulator mathematically cannot predict anything lower than ∼350µs, even though the real MPI implementation can push a 1KB message through the Pi's shared memory or fast-path network stack much faster.

2.7.3 The PPN (Processes Per Node) Bottleneck Trend

Notice what happens when you increase PPN (keeping Nodes and Message Size constant). Let's look at N=6, M=1048576 (1MB):

- P=1: Measured = 25,434 µs | Predicted = 54,736 µs (Sim is ∼2.1× higher)

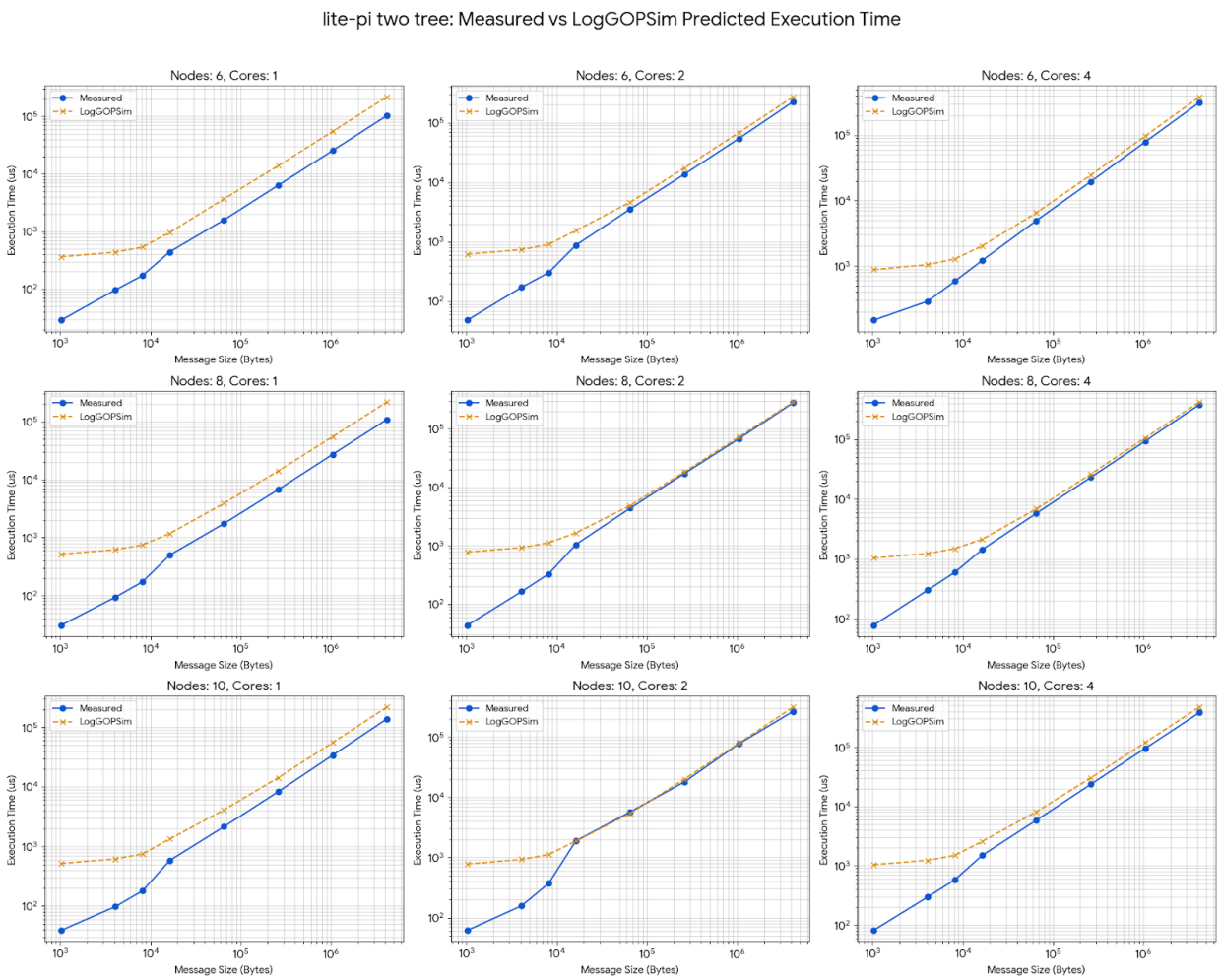


Figure 8: LogGOPS predictions vs. measured times on LITE-Pi cluster

- P=2: Measured = 54,253 µs | Predicted = 68,241 µs (Sim is ∼1.25× higher)
- P=4: Measured = 78,309 µs | Predicted = 96,254 µs (Sim is ∼1.22× higher)

Why does the prediction get more accurate as PPN increases? When $P = 1$, the network card is barely saturated, and the operation is CPU/Latency bound. As mentioned in the previous point, LogGOPSim is terrible at modeling pipelined latency. However, when $P = 4$, you have 4 processes on a single Raspberry Pi trying to blast data through a single Gigabit Ethernet NIC. The system becomes heavily bandwidth/injection bound. The LogGOP model is actually quite good at modeling strict bandwidth constraints and injection network contention. As contention becomes the dominant factor (overwhelming the static latency overheads), the simulator's theoretical bottleneck aligns much closer to the physical NIC's real bottleneck.

2.7.4 Linear Scaling of the G Parameter vs. Hardware Offloading

For very large messages (e.g., M=4194304 or 4MB):

- Measured (N=6, P=1): ∼103,136 µs
- Predicted (N=6, P=1): ∼218,124 µs

The simulator uses the linear formula for payload transfer: Time = Bytes × G. For 4MB, it calculates $4,194,304 \times 0.008$ µs $\approx 33,554$ µs per network link traversal. However, real Gigabit NICs on the Raspberry Pi use TSO (TCP Segmentation Offload) and GSO (Generic Segmentation Offload). The CPU just hands a massive 4MB buffer to the NIC once, and the hardware handles the framing and transmission instantly. LogGOPSim does not model hardware offloading; it assumes the CPU pays a strict linear tax for every byte sent, resulting in the ∼2× overestimation for large payloads.

2.7.5 Strict Trace Synchronization (MPI_Waitall)

The LogGOPSim engine plays back the schedgen goals exactly as recorded. When the trace dictates `MPI_Waitall`, the simulator completely halts the modeled process until all dependency messages arrive. In a real MPI implementation running on Linux, while the main thread is blocked in Waitall, a background progression thread (or interrupt handler) is actively pulling packets off the wire and advancing other pending non-blocking requests asynchronously. The simulator's rigid graph execution prevents this "hidden" overlap, resulting in stacked delays.

2.8 Summary of LogGOPS Behavior

The LogGOPSim results are showing the theoretical worst-case bounds of your algorithm if the network had zero pipelining, zero hardware offloading, and every message incurred the cost of a cold-start ping-pong. The measured results are much faster because the Raspberry Pi OS and MPI implementation are highly optimized to overlap CPU overhead with network transmission during collective operations.

2.9 Impact of Parameters: Nodes, PPN, and Message Size

LogGOPSim mathematically scales based on the tuple (Nodes, PPN, Message Size). Here is how each parameter alters the simulated behavior:

2.9.1 A. Number of Nodes (N)

Increasing the number of nodes increases the depth of the communication tree.

Trend: The execution time grows logarithmically (roughly $O(\log N)$).

Data Evidence (CDAC, M=4MB):

- N=8: 2.78 ms
- N=16: 4.93 ms

Explanation: `two_tree_bcast` uses pipelined binary trees. Doubling the nodes from 8 to 16 only adds 1 extra layer of depth to the tree, adding exactly one extra hop of latency and gap delay.

2.9.2 B. Message Size (M)

Increasing the message size directly stresses the bandwidth parameters (G and O).

Trend: The execution time grows linearly ($O(M)$).

Data Evidence (CDAC, N=16):

- 1KB: 48 µs
- 4MB: 4,937 µs

Explanation: For small messages (1KB), the time is entirely dominated by the network hardware latency (L). For large messages (4MB), latency becomes irrelevant, and the time is dominated by how fast bytes can be pushed through the Network Interface Card (NIC), dictated by the G parameter.

2.9.3 C. Processes Per Node (PPN)

Increasing PPN packs more MPI ranks onto the same physical machine. (Note: Our CDAC goal runs were PPN=1, but LITE Pi explores this).

Trend: Increasing PPN drastically increases the predicted execution time.

Data Evidence (LITE Pi, N=6, M=1KB):

- PPN=1: 362 µs
- PPN=4: 880 µs

Explanation: LogGOPSim is heavily penalized by PPN because all 4 processes on the same node must serialize their network injections through a single NIC. LogGOPSim models this using the g (gap) parameter. Instead of 1 process firing a message, 4 processes queue up, multiplying the injection delay by 4. (Note: In real-world measured time, PPN > 1 is often heavily accelerated by OS-level shared-memory copying, bypassing the NIC entirely, a nuance the standard LogGOP model does not natively capture).